Egypt-Japan University of Science and Technology

Faculty of Engineering — Department of Computer Science & Engineering

CSE323 — Software Engineering

Design Specification Report

Deliverable D3 — Phase 2

Sub-system: **Customer Ordering System**

Sprint 1 Vertical Slice: UC-3 | UC-4 | UC-5

Submitted to:

Dr. Rami Zewail
Eng. Samy Shaawat
Eng. Rowida Elsayed

Team Members:

#	Name	Student ID
1	Zeyad Ashraf Tawfik Mohamed Attia	120230010
2	Ahmed Mohamed Rashed	120230263
3	Ahmed Youssef Kamal	120230255
4	Samir Ahmed	120230292
5	Hussein Sabri Youssef	120230244

Contents

1. Gherkin Scripts

The following scripts cover the complete Sprint 1 vertical slice (UC-3, UC-4, UC-5). Every scenario is prefixed with a comment tagging the Requirement IDs it enforces, establishing a direct traceability link back to the D2 Requirements Report. `Scenario Outline` blocks are used wherever a guard must be verified across multiple input variations (REQ12, REQ13, REQ17, REQ18, REQ19).

1.1. Feature 1 — Add Item to Cart (UC-3)

Requirements covered: REQ3, REQ12, REQ13, REQ14, REQ20, REQ21

1.1.1. Background and Happy Path

```
Feature: UC-3 -- Add Item to Cart
  As an authenticated or guest user
  I want to select a menu item, specify a valid quantity, and add it to my cart
  So that I can proceed toward checkout with a well-formed, valid order

  Background:
    Given the Customer Ordering System is running and reachable
    And the following restaurants exist within their server-time operating window:
      | restaurant_id | name          | status |
      | R001          | Burger Palace | OPEN   |
      | R002          | Pizza Kingdom | OPEN   |
    And the following menu items exist in the system database:
      | item_id | restaurant_id | name                    | price_egp| available |
      | I001    | R001          | Classic Burger          | 75.00    | true      |
      | I002    | R001          | Crispy Chicken Sandwich | 85.00    | true      |
      | I003    | R001          | Unavailable Special     | 50.00    | false     |
      | I004    | R002          | Pepperoni Pizza         | 110.00   | true      |
    And the user has an active session (authenticated or guest with HTTP-only
    cookie)
    And the user's cart is empty

  # Covers: REQ3
  Scenario: Successfully add a valid available item to an empty cart
    Given the user is viewing the menu for "Burger Palace"
    When the user selects item "Classic Burger" (item_id: I001)
    And the user sets the quantity to 2
    And the user clicks "Add to Cart"
    Then the system shall add 2 units of "Classic Burger" at 75.00 EGP each
    And the cart item count displayed in the UI shall update to 1 line item
    And the cart subtotal displayed shall equal 150.00 EGP
    And no full page reload shall occur during this interaction
```

Listing 1: UC-3 Feature declaration, Background, and Happy Path

1.1.2. REQ12 — Negative and Zero Quantity Guard

```
# Covers: REQ12
Scenario Outline: Reject zero or negative integer quantity at UI layer
  Given the user is viewing the menu for "Burger Palace"
  And the user selects item "Classic Burger" (item_id: I001)
  When the user enters <invalid_qty> into the quantity field
  Then the "Add to Cart" submit button shall be disabled immediately
  And no HTTP request shall be dispatched to the cart API endpoint
  And a UI-level message shall read "Quantity must be at least 1"

  Examples:
    | invalid_qty | description    |
    | 0           | zero boundary  |
    | -1          | negative one   |
    | -100        | large negative |

# Covers: REQ12
Scenario Outline: Reject zero or negative quantity at server-side API layer
  Given a malicious client bypasses the UI and sends POST to
 "/api/v1/cart/items"
  When the request body contains:
    | field    | value          |
    | item_id  | I001           |
    | quantity | <invalid_qty> |
  Then the server shall respond with HTTP 422 Unprocessable Entity
  And the response body error shall equal "Quantity must be at least 1"
  And no cart record shall be created or modified in the database

  Examples:
    | invalid_qty | description   |
    | 0           | zero boundary |
    | -5          | negative five |
```

Listing 2: REQ12: Negative / Zero Quantity Guards at UI and Server layers

1.1.3. REQ13 — Fractional Quantity Guard

```
# Covers: REQ13
Scenario Outline: Reject fractional or decimal quantity at UI layer
  Given the user is viewing the menu for "Burger Palace"
  And the user selects item "Classic Burger" (item_id: I001)
  When the user attempts to enter <decimal_qty> into the quantity field
  Then the quantity input field shall not accept a decimal point character
  And the "Add to Cart" button shall remain disabled

  Examples:
    | decimal_qty | description         |
    | 0.5         | half unit           |
    | 2.7         | fractional quantity |
    | 1.0         | trailing zero float |
```

```
  # Covers: REQ13
  Scenario Outline: Reject fractional quantity at server-side API layer
    Given a client sends POST to "/api/v1/cart/items" with a decimal quantity
    When the request body contains:
      | field    | value         |
      | item_id  | I001          |
      | quantity | <decimal_qty> |
    Then the server shall respond with HTTP 422 Unprocessable Entity
    And the response body error shall equal "Quantity must be a whole number"

    Examples:
      | decimal_qty | description     |
      | 0.5         | half unit       |
      | 2.3         | two point three |
      | NaN         | not-a-number    |
```

Listing 3: REQ13: Fractional / Decimal Quantity Guards at UI and Server layers

1.1.4. REQ14 — Cross-Restaurant Cart Guard

```
  # Covers: REQ14
  Scenario: Prompt user when adding an item from a different restaurant
    Given the cart contains "Classic Burger" from "Burger Palace" (R001)
    When the user selects "Pepperoni Pizza" from "Pizza Kingdom" (R002)
    And the user clicks "Add to Cart"
    Then the system shall display a conflict modal before any cart mutation
    And the modal text shall include:
        "Adding items from a new restaurant will clear your current cart."
    And the modal shall present "Yes, clear cart and add" and "Cancel"

  # Covers: REQ14
  Scenario: User confirms cross-restaurant replacement
    Given the cross-restaurant conflict modal is displayed
    When the user clicks "Yes, clear cart and add"
    Then the system shall discard all items from "Burger Palace"
    And shall add "Pepperoni Pizza" (qty: 1) to the now-empty cart

  # Covers: REQ14
  Scenario: User cancels cross-restaurant replacement
    Given the cross-restaurant conflict modal is displayed
    When the user clicks "Cancel"
    Then the cart shall retain its original contents from "Burger Palace"
    And no items from "Pizza Kingdom" shall appear in the cart
```

Listing 4: REQ14: Cross-Restaurant Cart Conflict scenarios

1.1.5. REQ20 and REQ21 — Unavailable Item and Empty Cart Guards

```
  # Covers: REQ20
  Scenario: Block adding an item marked unavailable in the database
    Given item "Unavailable Special" (I003) has availability = false
    When the user attempts to click on "Unavailable Special"
```

```
    Then the item tile shall be displayed in a greyed-out non-interactive state
    And no POST request shall be sent to the cart endpoint for item_id I003

  # Covers: REQ21
  Scenario: Checkout button is disabled when cart is empty
    Given the user has not added any items to the cart
    When the user navigates to the cart page
    Then the "Proceed to Checkout" button shall be in a disabled state
    And clicking the disabled button shall produce no navigation or HTTP request
    And the UI shall display "Your cart is empty. Add items to continue."
```

Listing 5: REQ20 and REQ21: Item unavailability and empty cart guard

1.2. Feature 2 — Manage Shopping Cart (UC-4)

Requirements covered: REQ4, REQ15, REQ21

```
Feature: UC-4 -- Manage Shopping Cart
  As an authenticated or guest user
  I want to view, modify, and confirm the contents of my cart
  So that I can ensure my order is correct before initiating checkout

  Background:
    Given the Customer Ordering System is running and reachable
    And the user has an active session
    And the user's cart contains the following items:
      | line_item_id | item_id | name                    | qty | unit_price_egp |
      | L001         | I001    | Classic Burger          | 2   | 75.00          |
      | L002         | I002    | Crispy Chicken Sandwich | 1   | 85.00          |
    And the cart subtotal at session creation time is 235.00 EGP

  # Covers: REQ4
  Scenario: Display all cart items with correct quantities, prices, and subtotal
    Given the user navigates to the cart page
    Then the system shall render a cart interface containing 2 line items
    And each line item shall display name, unit price, quantity, and line total
    And the cart subtotal shall display 235.00 EGP
    And this page shall render without a full page reload

  # Covers: REQ4
  Scenario: Update quantity and verify real-time subtotal recalculation
    Given the user is viewing the cart page
    When the user changes quantity of "Classic Burger" (L001) from 2 to 3
    Then the line total for "Classic Burger" shall update to 225.00 EGP
    And the cart subtotal shall update to 310.00 EGP without a full page reload
    And the database cart record for L001 shall reflect quantity = 3

  # Covers: REQ4
  Scenario: Remove a single item from the cart
    Given the user is viewing the cart page
    When the user clicks "Remove" for "Crispy Chicken Sandwich" (L002)
    Then that item shall be removed without a full page reload
```

```
    And the cart shall contain exactly 1 remaining line item
    And the cart subtotal shall update to 150.00 EGP
```

Listing 6: UC-4 Background and cart review / modification scenarios

```
  # Covers: REQ15
  Scenario: Stale prices refreshed to live database values before checkout renders
    Given "Classic Burger" was added 4 hours ago at a cached price of 75.00 EGP
    And the restaurant has since updated its price to 90.00 EGP in the database
    When the user clicks "Proceed to Checkout"
    Then the system shall re-query current price of every cart item from the DB
    And the checkout screen shall display "Classic Burger" at 90.00 EGP
    And the checkout total shall reflect 180.00 EGP for 2 units
    And "Confirm Order" shall only be enabled after the refresh completes

  # Covers: REQ15, REQ20
  Scenario: Checkout halts when refresh detects an unavailable item
    Given the cart contains "Crispy Chicken Sandwich" (I002)
    And the database has since set item I002 availability to false
    When the user clicks "Proceed to Checkout"
    Then the checkout screen shall NOT render
    And the notification shall read "Crispy Chicken Sandwich is no longer
   available"
    And "Confirm Order" shall remain disabled until the user removes item I002

  # Covers: REQ21
  Scenario: Server-side empty cart guard rejects checkout with zero line items
    Given a client sends POST to "/api/v1/orders/checkout" with empty cart session
    When the server processes the request
    Then the server shall verify |cart.items| >= 1 before any checkout processing
    And shall respond with HTTP 400 Bad Request
    And the response error shall equal "Cart is empty"
    And no payment request shall be initiated
```

Listing 7: REQ15 and REQ21: Stale-cart refresh and server-side empty cart guard

1.3. Feature 3 — Secure Checkout and Payment (UC-5)

Requirements covered: REQ7, REQ16, REQ17, REQ18, REQ19, REQ20

1.3.1. Happy Path and Idempotent Submission Guard

```
Feature: UC-5 -- Secure Checkout and Payment
  As an authenticated or guest user
  I want to confirm my order and process payment securely
  So that a verified, correctly-priced order is created and dispatched

  Background:
    Given the Customer Ordering System is running and reachable
    And the user has an active session
    And the cart contains item I001 "Classic Burger" qty 2, current price 75.00
   EGP
```

```
    And the server-computed total is 150.00 EGP
    And the server UTC time is within the operating window of "Burger Palace"
    And the checkout confirmation screen is displayed

  # Covers: REQ7
  Scenario: Successful end-to-end checkout creates confirmed order on payment
   success
    Given the checkout screen shows the server-computed total of 150.00 EGP
    When the user clicks "Confirm Order"
    Then the system shall disable the button synchronously before any HTTP
   dispatch
    And shall submit a payment authorisation to the Gateway for 150.00 EGP
    And upon a success callback shall create a confirmed order with status
   CONFIRMED
    And the order record shall store total_egp = 150.00 and a payment_reference

  # Covers: REQ7
  Scenario: Order record is NOT created when gateway returns failure
    Given the checkout screen is displayed
    When the user clicks "Confirm Order"
    And the Payment Gateway returns a payment failure response
    Then the system shall NOT create any order record in the database
    And "Confirm Order" shall be re-enabled to allow the user to retry

  # Covers: REQ16
  Scenario: Button disabled synchronously on first click before HTTP dispatch
    Given the checkout confirmation screen is displaying the order summary
    When the user clicks "Confirm Order" for the first time
    Then the button shall transition to disabled and loading state synchronously
        within the same JavaScript event loop tick before any HTTP request fires
    And it shall remain disabled until a terminal server response is received

  # Covers: REQ16
  Scenario: Rapid successive clicks produce exactly one POST request
    Given the checkout confirmation screen is displaying the order summary
    When the user clicks "Confirm Order" three times within 200 ms
    Then exactly one POST shall be dispatched to "/api/v1/orders/checkout"
    And the database shall contain exactly one order record for this session
    And no duplicate payment authorisation shall reach the Payment Gateway
```

Listing 8: UC-5 Background, happy path, and REQ16 idempotency guard

1.3.2. REQ17 — Server-Side Price Recalculation Guard

```
  # Covers: REQ17
  Scenario: Server discards client-supplied total and recalculates from database
    Given the checkout POST body contains client_total_egp: 0.01
    And the cart contains item I001 qty 2 with database price 75.00 EGP
    When the server receives the checkout POST
    Then the server shall independently compute total as (2 x 75.00) = 150.00 EGP
    And the Gateway shall be charged 150.00 EGP and NOT 0.01 EGP
    And the confirmed order record shall store total_egp = 150.00
```

```
# Covers: REQ17
Scenario Outline: Server recalculation is correct regardless of client-supplied
 value
  Given the client sends a checkout POST with manipulated total <manipulated>
  And the cart contains item I001, qty <qty>
  And the current database price of I001 is <db_price> EGP
  When the server processes the checkout
  Then the server-computed total shall equal <expected> EGP
  And the payment charge shall be exactly <expected> EGP

  Examples:
    | manipulated | qty | db_price | expected |
    | 0.01        | 2   | 75.00    | 150.00   |
    | 999999.00   | 1   | 85.00    | 85.00    |
    | -50.00      | 3   | 75.00    | 225.00   |
```

Listing 9: REQ17: Price manipulation guard scenarios

1.3.3. REQ18 — DoS-Prevention Character Limit Guard

```
# Covers: REQ18
Scenario: UI enforces 500-character limit on Special Instructions field
  Given the user is on the checkout confirmation screen
  When the user pastes a 600-character string into "Special Instructions"
  Then the field shall accept a maximum of 500 characters
  And the HTML maxlength attribute on this field shall equal 500
  And the character counter shall display "500/500"

# Covers: REQ18
Scenario: Server rejects Special Instructions exceeding 500 characters
  Given a client bypasses the UI and sends POST to "/api/v1/orders/checkout"
  And the "special_instructions" field contains a 501-character string
  When the server validates the request
  Then the server shall respond with HTTP 422 Unprocessable Entity
  And the error shall read "Special instructions must not exceed 500 characters"
  And no order record or payment request shall be created

# Covers: REQ18
Scenario Outline: Server enforces 500-char limit across all free-text fields
  Given a client sends POST with field "<field_name>" of length <chars>
  When the server validates the payload
  Then the response status shall be <status>

  Examples:
    | field_name           | chars | status |
    | special_instructions | 500   | 200    |
    | special_instructions | 501   | 422    |
    | delivery_notes       | 500   | 200    |
    | delivery_notes       | 501   | 422    |
```

Listing 10: REQ18: 500-character limit on free-text fields

1.3.4. REQ19 — Server-Time Operating-Hours Validation

```
# Covers: REQ19
Scenario: Checkout rejected when restaurant is closed at server UTC time
  Given "Burger Palace" has operating hours 10:00-22:00 server UTC
  And the current server UTC time is 03:00 (outside operating hours)
  And the user's system clock is set to 14:00 (spoofed client time)
  When the user submits the checkout POST
  Then the server shall evaluate hours using its own UTC clock only
  And shall respond with HTTP 403 Forbidden
  And the message shall read "Burger Palace is currently closed"
  And no payment request or order record shall be created

# Covers: REQ19
Scenario Outline: Operating-hours gate enforced at exact boundary cases
  Given "Burger Palace" has operating hours 10:00-22:00 server UTC
  When a checkout request arrives at server UTC time <server_time>
  Then the checkout response status shall be <status>
  And an order shall be created only if <created> is true

  Examples:
    | server_time | status | created | description                |
    | 09:59       | 403    | false   | 1 minute before opening    |
    | 10:00       | 200    | true    | exact opening boundary     |
    | 21:59       | 200    | true    | last valid minute          |
    | 22:00       | 403    | false   | exact closing boundary     |
    | 03:00       | 403    | false   | middle of the night        |
```

Listing 11: REQ19: Server-UTC operating-hours gate at checkout

1.3.5. REQ20 — Unavailable Item Block at Checkout Stage

```
# Covers: REQ20, REQ15
Scenario: Checkout halts when pre-checkout refresh detects unavailable item
  Given the cart contains "Crispy Chicken Sandwich" (I002)
  And the database has set item I002 availability to false
  When the pre-checkout cart refresh executes on "Proceed to Checkout"
  Then the checkout flow shall halt before the "Confirm Order" button renders
  And the notification shall read "Crispy Chicken Sandwich is no longer
   available"
  And "Confirm Order" shall not be enabled until the user removes I002

# Covers: REQ20
Scenario: Server-side block rejects checkout even if client-side check is
  bypassed
  Given a client bypasses the UI and sends POST to "/api/v1/orders/checkout"
  And the payload includes item_id I002 which has availability = false
  When the server processes the checkout request
  Then the server shall re-query availability for all submitted items
  And shall respond with HTTP 422 Unprocessable Entity
  And the error shall list "Crispy Chicken Sandwich (I002) is not available"
  And no order record shall be created and no payment shall be dispatched
```

Listing 12: REQ20: Checkout block for unavailable items detected at checkout stage

2. QA Refinement Loop — Senior QA Audit Log

2.1. REQ4: Cart Responsiveness

Feature: UC-4 — Manage Shopping Cart | **Vague term eliminated:** *"instantly"*

[DRAFT — VAGUE] *"The cart should update instantly when the user changes a quantity or removes an item."*

[QA CRITIQUE]

- **"Instantly" is a relative, unmeasurable adjective.** A Playwright assertion cannot query "instantaneousness." The term defines no threshold, no mechanism, and no observable DOM state. A 3-second re-render could satisfy a human reading of "instant" while violating any real performance contract. There is no boolean outcome to assert against.
- No observable DOM state change is specified for the test runner to detect.

[REFINED METRIC — AS USED IN GHERKIN]

```
# Covers: REQ4
Scenario: Update quantity and verify real-time subtotal recalculation
  Given the user is viewing the cart page
  When the user changes quantity of "Classic Burger" (L001) from 2 to 3
  Then the line total for "Classic Burger" shall update to 225.00 EGP
  And the cart subtotal shall update to 310.00 EGP without a full page reload
  And the database cart record for L001 shall reflect quantity = 3
```

Listing 13: REQ4 Refined: "No full page reload" as the measurable DOM assertion

Metric adopted: Cart DOM elements must update **without a full page reload** — verified by asserting no `DOMContentLoaded` or `load` navigation event fires, and the updated subtotal value is present in the existing DOM within the same page lifecycle.

2.2. REQ16: Idempotent Order Submission

Feature: UC-5 — Secure Checkout | **Vague term eliminated:** *"prevent double-clicking"*

[DRAFT — VAGUE] *"The system should prevent the user from double-clicking the Confirm Order button to avoid duplicate orders."*

[QA CRITIQUE]

- **"Should prevent" defines neither the mechanism nor the timing boundary.** A

naïve implementation could disable the button 300 ms after click — after two HTTP requests have already been dispatched.

- **"Double-clicking" is too narrow.** Under network latency, rapid successive single-clicks are the real attack; "double-click" misses this entirely.
- No concrete DOM state change or network-level assertion is specified. No boolean outcome exists for Playwright to evaluate.

[REFINED METRIC — AS USED IN GHERKIN]

```
# Covers: REQ16
Scenario: Button disabled synchronously on first click before HTTP dispatch
  Given the checkout confirmation screen is displaying the order summary
  When the user clicks "Confirm Order" for the first time
  Then the button shall transition to disabled and loading state synchronously
      within the same JavaScript event loop tick before any HTTP request fires
  And it shall remain disabled until a terminal server response is received

# Covers: REQ16
Scenario: Rapid successive clicks produce exactly one POST request
  Given the checkout confirmation screen is displaying the order summary
  When the user clicks "Confirm Order" three times within 200 ms
  Then exactly one POST shall be dispatched to "/api/v1/orders/checkout"
  And the database shall contain exactly one order record for this session
  And no duplicate payment authorisation shall reach the Payment Gateway
```

Listing 14: REQ16 Refined: Synchronous button disable within same JS event loop tick

Metric adopted: Button must have `disabled=true` **synchronously within the same JavaScript event loop tick as the click event** — before any `fetch()` is dispatched. Verified via `page.route()`: the `disabled` attribute must be `true` at the moment the first intercepted network request fires. Exactly **one** POST to `/api/v1/orders/checkout` per checkout session.

2.3. REQ17: Server-Side Price Integrity

Feature: UC-5 — Secure Checkout | **Vague term eliminated:** *"trusted and secure price"*

[DRAFT — VAGUE] *"Make sure the price sent by the client is trusted and secure during checkout."*

[QA CRITIQUE]

- **"Trusted" and "secure" are architectural aspirations, not testable assertions.** This phrasing dangerously implies the server may use the client-supplied price as long as it arrives over HTTPS.
- It identifies no data flow, no discard mechanism, no computation source, and no observable output that a test runner can verify.
- It would pass a code review while leaving the price-manipulation attack vector entirely open.

[REFINED METRIC — AS USED IN GHERKIN]

```
  # Covers: REQ17
  Scenario: Server discards client-supplied total and recalculates from database
    Given the checkout POST body contains client_total_egp: 0.01
    And the cart contains item I001 qty 2 with database price 75.00 EGP
    When the server receives the checkout POST
    Then the server shall independently compute total as (2 x 75.00) = 150.00 EGP
    And the Gateway shall be charged 150.00 EGP and NOT 0.01 EGP
    And the confirmed order record shall store total_egp = 150.00
```

Listing 15: REQ17 Refined: Server silently discards client total, charges DB-computed amount

Metric adopted: The server must **silently discard `client_total_egp`** and independently recompute the total as $\sum(\text{db\_price}[i] \times \text{qty}[i])$. Verified by sending `client_total_egp: 0.01` for a cart worth 150.00 EGP — the Payment Gateway mock must receive exactly **150.00 EGP** and the order record must store `total_egp = 150.00`.

2.4. REQ12 / REQ13: Quantity Input Validation

Feature: UC-3 — Add Item to Cart | **Vague term eliminated:** *"invalid quantities shouldn't work"*

[DRAFT — VAGUE] *"Invalid quantities shouldn't be allowed when adding items to the cart."*

[QA CRITIQUE]

- **"Invalid" is undefined and circular.** This phrasing answers nothing: invalid according to what type system? What domain? Does `0` count? Does `2.0`?
- No HTTP status code, no error message string, and no observable UI state are specified. A Playwright test cannot assert against an abstract concept.

[REFINED METRIC — AS USED IN GHERKIN]

```
  # Covers: REQ12
  Scenario Outline: Reject zero or negative integer quantity at server-side API
   layer
    Given a malicious client bypasses the UI and sends POST to
   "/api/v1/cart/items"
    When the request body contains:
      | field    | value         |
      | item_id  | I001          |
      | quantity | <invalid_qty> |
    Then the server shall respond with HTTP 422 Unprocessable Entity
    And the response body error shall equal "Quantity must be at least 1"
    And no cart record shall be created or modified in the database

    Examples:
      | invalid_qty | description   |
      | 0           | zero boundary |
      | -5          | negative five |

  # Covers: REQ13
```

```
  Scenario Outline: Reject fractional quantity at server-side API layer
    Given a client sends POST to "/api/v1/cart/items" with a decimal quantity
    When the request body contains:
      | field    | value         |
      | item_id  | I001          |
      | quantity | <decimal_qty> |
    Then the server shall respond with HTTP 422 Unprocessable Entity
    And the response body error shall equal "Quantity must be a whole number"

    Examples:
      | decimal_qty | description     |
      | 0.5         | half unit       |
      | NaN         | not-a-number    |
```

Listing 16: REQ12/13 Refined: Strict positive integer type, HTTP 422 with specific message

Metric adopted: Quantity field must accept only **strict positive integers** (`quantity` $\in \mathbb{Z}^+$). UI: submit button `disabled` while value ≤ 0 or contains a decimal. API: any violation returns **HTTP 422** with either `"Quantity must be at least 1"` or `"Quantity must be a whole number"`.

2.5. REQ18: DoS-Prevention Character Limit

Feature: UC-5 — Secure Checkout | **Vague term eliminated:** *"don't allow very long text"*

[DRAFT — VAGUE] *"Don't allow users to enter very long text into the Special Instructions field to prevent abuse."*

[QA CRITIQUE]

- **"Very long" is unquantifiable and inconsistent.** Without a concrete number, two developers will implement different limits (`VARCHAR(1000)` vs. `VARCHAR(255)`), and no automated test can assert "too long" as a boolean condition.
- **"Prevent abuse"** describes an intent, not an observable system behaviour. The assertion requires an exact integer boundary.

[REFINED METRIC — AS USED IN GHERKIN]

```
  # Covers: REQ18
  Scenario: Server rejects Special Instructions exceeding 500 characters
    Given a client bypasses the UI and sends POST to "/api/v1/orders/checkout"
    And the "special_instructions" field contains a 501-character string
    When the server validates the request
    Then the server shall respond with HTTP 422 Unprocessable Entity
    And the error shall read "Special instructions must not exceed 500 characters"
    And no order record or payment request shall be created

  # Covers: REQ18
  Scenario Outline: Server enforces 500-char limit across all free-text fields
    Given a client sends POST with field "<field_name>" of length <chars>
    When the server validates the payload
```

```
  Then the response status shall be <status>

  Examples:
    | field_name           | chars | status |
    | special_instructions | 500   | 200    |
    | special_instructions | 501   | 422    |
    | delivery_notes       | 500   | 200    |
    | delivery_notes       | 501   | 422    |
```

Listing 17: REQ18 Refined: Hard 500-char limit, HTTP 200 at boundary, HTTP 422 beyond it

Metric adopted: All free-text fields enforce a **hard limit of exactly 500 characters**. UI: HTML `maxlength="500"` asserted via DOM attribute; paste truncated to 500. API: length > 500 returns **HTTP 422**; exactly 500 characters returns **HTTP 200**.

2.6. REQ19: Restaurant Operating-Hours Validation

Feature: UC-3, UC-4, UC-5 — All Sprint Slice Features | **Vague term eliminated:** *"check if restaurant is open"*

[DRAFT — VAGUE] *"The system should check if the restaurant is open before allowing an order to be placed."*

[QA CRITIQUE]

- **"Check if open" is silent on the authoritative time source** — which is the entire attack surface. A client clock is fully under user control; setting it to 14:00 while the server reads 03:00 trivially bypasses any client-side "open" check.
- No boundary times, no HTTP response code for a rejected request, and no definition of which clock governs are provided.
- A Playwright test cannot assert "checked the right clock" without a concrete observable outcome (HTTP status code, response body field).

[REFINED METRIC — AS USED IN GHERKIN]

```
# Covers: REQ19
Scenario: Checkout rejected when restaurant is closed at server UTC time
  Given "Burger Palace" has operating hours 10:00-22:00 server UTC
  And the current server UTC time is 03:00 (outside operating hours)
  And the user's system clock is set to 14:00 (spoofed client time)
  When the user submits the checkout POST
  Then the server shall evaluate hours using its own UTC clock only
  And shall respond with HTTP 403 Forbidden
  And the message shall read "Burger Palace is currently closed"
  And no payment request or order record shall be created

# Covers: REQ19
Scenario Outline: Operating-hours gate enforced at exact boundary cases
  Given "Burger Palace" has operating hours 10:00-22:00 server UTC
  When a checkout request arrives at server UTC time <server_time>
  Then the checkout response status shall be <status>
```

```
    And an order shall be created only if <created> is true

    Examples:
      | server_time | status | created | description              |
      | 09:59       | 403    | false   | 1 minute before opening  |
      | 10:00       | 200    | true    | exact opening boundary   |
      | 21:59       | 200    | true    | last valid minute        |
      | 22:00       | 403    | false   | exact closing boundary   |
      | 03:00       | 403    | false   | middle of the night      |
```

Listing 18: REQ19 Refined: Server UTC clock only, HTTP 403, boundary Examples table

Metric adopted: Operating-hours eligibility must be evaluated **exclusively using the server's UTC clock**. At checkout, a request outside the defined window returns **HTTP 403** with `"<Name> is currently closed"`. Verified by a Playwright test that spoofs the client system clock to a valid time while the *server* clock is set to an out-of-hours UTC timestamp — expected outcome is **HTTP 403** regardless of client-side time.

3. API Contracts and Information Hiding

Table 1: Global API Conventions

Convention	Value
Base URL	`/api/v1`
Content-Type	`application/json`
Authentication	HTTP-only session cookie (`session_id`)
Monetary values	`number` with exactly 2 decimal places, in EGP
Timestamps	ISO 8601 UTC strings: `"2026-05-10T14:32:00Z"`
Error envelope	Uniform structure across all endpoints (see below)

```
{
  "error": {
    "code":    "VALIDATION_ERROR",
    "message": "Human-readable description of the error.",
    "fields": {
      "field_name": "Field-specific detail (present only for validation errors)"
    }
  }
}
```

Listing 19: Standard Error Envelope — applies to all non-2xx responses

3.1. Endpoint 1 — Add Item to Cart

Method & URL	`POST /api/v1/cart/items`
Maps to	UC-3: Add Item to Cart
Enforces	REQ3, REQ12, REQ13, REQ14, REQ20

Purpose. Appends a single menu item at a specified quantity to the active session cart. The server enforces all cart integrity invariants (quantity domain, cross-restaurant conflict, item availability) before mutating cart state. The client supplies only `item_id` and `quantity`; all business rules are evaluated server-side.

```
{
  "item_id":  "string",   // Required. Must reference an existing menu item.
  "quantity": "integer"   // Required. Strict positive integer >= 1.
                          // REQ12: 0 and negatives rejected with HTTP 422.
                          // REQ13: Decimals and NaN rejected with HTTP 422.
}
```

Listing 20: POST /api/v1/cart/items — Request Payload

```
{
  "cart": {
    "cart_id":         "cart_abc123",
    "restaurant_id":   "R001",
    "restaurant_name": "Burger Palace",
    "items": [
      {
        "line_item_id":   "L001",
        "item_id":        "I001",
        "name":           "Classic Burger",
        "quantity":       2,
        "unit_price_egp": 75.00,
        "line_total_egp": 150.00
      }
    ],
    "subtotal_egp": 150.00,
    "item_count":   1
  }
}
```

Listing 21: HTTP 200 – Item Successfully Added

```
{
  "error": {
    "code":    "CROSS_RESTAURANT_CONFLICT",
    "message": "Your cart contains items from a different restaurant.",
    "conflict": {
      "current_restaurant_id":     "R001",
      "current_restaurant_name":   "Burger Palace",
      "requested_restaurant_id":   "R002",
      "requested_restaurant_name": "Pizza Kingdom"
```

```
    }
  }
}
```

Listing 22: HTTP 409 – Cross-Restaurant Cart Conflict (REQ14)

```
{
  "error": {
    "code":    "VALIDATION_ERROR",
    "message": "Request payload failed validation.",
    "fields": {
      "quantity": "Quantity must be a whole number greater than or equal to 1."
    }
  }
}
```

Listing 23: HTTP 422 – Quantity Validation Failure (REQ12 / REQ13)

```
{
  "error": {
    "code":    "ITEM_UNAVAILABLE",
    "message": "The requested item is not currently available.",
    "fields": {
      "item_id": "Classic Burger is not currently available."
    }
  }
}
```

Listing 24: HTTP 422 – Item Unavailable (REQ20)

Information Hiding Rationale

- **Database schema** — the response exposes `line_item_id` and computed totals but reveals nothing about the storage mechanism. Team B (Checkout) reads `cart_id` without querying the cart database directly.
- **Cross-restaurant detection** — the `restaurant_id` comparison is entirely internal. The client receives `HTTP 409` and reacts; the algorithm is hidden behind the response code.
- **Pricing authority** — the server populates `unit_price_egp` from the database. The client never sends a price, making it structurally impossible to submit one.
- **Availability mechanism** — the client has no knowledge of how availability is evaluated; the interface exposes only the outcome (`HTTP 422`, `ITEM_UNAVAILABLE`).

3.2. Endpoint 2 — Retrieve and Refresh Cart

Method & URL	`GET /api/v1/cart`
Maps to	UC-4: Manage Shopping Cart
Enforces	REQ4, REQ15, REQ21

Purpose. Returns the session cart with all prices re-queried from the live database at the moment of the call. Serves both the cart-review UI (REQ4) and the pre-checkout stale-cart refresh (REQ15). Unavailable items are flagged rather than silently removed, enabling explicit user notification (REQ20). No request body is required.

```
{
  "cart": {
    "cart_id":          "cart_abc123",
    "restaurant_id":    "R001",
    "restaurant_name": "Burger Palace",
    "items": [
      {
        "line_item_id":    "L001",
        "item_id":         "I001",
        "name":            "Classic Burger",
        "quantity":        2,
        "unit_price_egp": 90.00,     // Refreshed from cached 75.00 EGP (REQ15)
        "line_total_egp": 180.00,
        "available":       true,
        "price_updated":   true      // Signals UI to highlight the price change
      },
      {
        "line_item_id":    "L002",
        "item_id":         "I002",
        "name":            "Crispy Chicken Sandwich",
        "quantity":        1,
        "unit_price_egp": 85.00,
        "line_total_egp": 85.00,
        "available":       false,    // REQ20: user must be notified explicitly
        "price_updated":   false
      }
    ],
    "subtotal_egp":       265.00,
    "item_count":         2,
    "checkout_eligible": false,     // REQ21: false because available=false exists
    "unavailable_items": ["I002"]
  }
}
```

Listing 25: HTTP 200 – Cart Retrieved (stale price updated; one item flagged unavailable)

```
{
  "cart": {
    "cart_id":            "cart_abc123",
    "restaurant_id":      null,
    "restaurant_name":    null,
    "items":              [],
    "subtotal_egp":       0.00,
    "item_count":         0,
    "checkout_eligible": false,
    "unavailable_items": []
  }
```

```
}
```

Listing 26: HTTP 200 – Empty Cart (valid state, not an error)

Information Hiding Rationale

- **Refresh mechanism** — the client has no knowledge that a database re-query occurs on every `GET`. The cache-invalidation strategy and any caching layer are fully opaque.
- **Eligibility computation** — `checkout_eligible` encapsulates two independent business rules. The client reads one boolean and sets button state without re-implementing any logic in JavaScript.
- **Session identity** — cart retrieval via cookie prevents cart enumeration; no client-supplied cart identifier exists.

3.3. Endpoint 3 — Initiate Secure Checkout

Method & URL	`POST /api/v1/orders/checkout`
Maps to	UC-5: Secure Checkout & Payment
Enforces	REQ7, REQ16, REQ17, REQ18, REQ19, REQ20, REQ21

Purpose. Executes the complete atomic checkout pipeline in a single request: validation → price recalculation → operating-hours gate → Payment Gateway authorisation → order persistence. Returns a confirmed order only on full pipeline success; any failure aborts the pipeline with no side effects.

```
{
  "delivery_address": {
    "street": "string",      // Required. Max 200 characters.
    "city":   "string",      // Required. Max 100 characters.
    "notes":  "string|null"  // Optional. Max 500 characters (REQ18).
  },
  "payment_method": {
    "gateway_token": "string" // Required. Opaque token from Payment Gateway SDK.
  },
  "client_total_egp":     "number",       // Accepted, then IMMEDIATELY DISCARDED.
                                           // Server recalculates from DB (REQ17).
  "special_instructions": "string|null"   // Optional. Hard max: 500 chars
    (REQ18).
}
```

Listing 27: POST /api/v1/orders/checkout — Request Payload

Security Invariant — REQ17: `client_total_egp` is received and immediately discarded. The server computes the authoritative total as $\sum(\text{db\_price}[i] \times \text{qty}[i])$ across all cart items. This invariant is non-optional on every request regardless of origin.

```
{
  "order": {
    "order_id":                  "ORD-20260510-00481",
```

```
    "status":                    "CONFIRMED",
    "restaurant_id":             "R001",
    "restaurant_name":           "Burger Palace",
    "items": [
      {
        "item_id":        "I001",
        "name":           "Classic Burger",
        "quantity":       2,
        "unit_price_egp": 75.00,
        "line_total_egp": 150.00
      }
    ],
    "server_computed_total_egp": 150.00,  // Never mirrors client_total_egp
    (REQ17)
    "delivery_address": {
      "street": "15 El-Geish Street, Mansoura",
      "city":   "Mansoura",
      "notes":  "Ring doorbell twice."
    },
    "special_instructions": "No onions on the burger, please.",
    "payment_reference":    "PAY-GW-TXN-77241",
    "created_at":           "2026-05-10T14:32:00Z"  // Server-generated UTC
  }
}
```

Listing 28: HTTP 201 – Order Confirmed (Happy Path)

```
{
  "error": {
    "code":    "CART_EMPTY",
    "message": "Cart is empty. Add items before initiating checkout."
  }
}
```

Listing 29: HTTP 400 – Empty Cart Pre-Condition Failure (REQ21)

```
{
  "error": {
    "code":    "RESTAURANT_CLOSED",
    "message": "Burger Palace is currently closed and cannot accept orders.",
    "details": {
      "restaurant_id":           "R001",
      "operating_hours_utc": {
        "open":  "10:00",
        "close": "22:00"
      },
      "server_utc_time_at_request": "03:00"
    }
  }
}
```

Listing 30: HTTP 403 – Restaurant Closed at Server UTC Time (REQ19)

```
{
  "error": {
    "code":                 "PAYMENT_FAILED",
    "message":              "Payment was unsuccessful. Please try again.",
    "gateway_decline_code":"insufficient_funds"
  }
}
```

Listing 31: HTTP 402 – Payment Gateway Failure

```
{
  "error": {
    "code":    "VALIDATION_ERROR",
    "message": "Request payload failed validation.",
    "fields": {
      "special_instructions":
          "Must not exceed 500 characters. Received: 743.",
      "delivery_address.notes":
          "Must not exceed 500 characters. Received: 612."
    }
  }
}
```

Listing 32: HTTP 422 – Field Validation Failure (REQ18)

```
{
  "error": {
    "code":    "ITEMS_UNAVAILABLE",
    "message": "One or more items in your cart are no longer available.",
    "unavailable_items": [
      {
        "item_id": "I002",
        "name":    "Crispy Chicken Sandwich",
        "reason":  "Item has been marked unavailable by the restaurant."
      }
    ]
  }
}
```

Listing 33: HTTP 422 – Unavailable Items Detected at Checkout (REQ20)

Information Hiding Rationale

- **Payment gateway integration** — the client submits an opaque `gateway_token` from the gateway's client-side SDK. API keys, endpoint URL, and the raw gateway response are hidden. The interface boundary is: token in, `payment_reference` out.
- **Atomic pipeline** — the client sends one request and receives one outcome with no knowledge of the five internal pipeline stages. The pipeline can be refactored without altering this contract.
- **Price computation authority** — naming `client_total_egp` as "received and immediately discarded" makes the trust boundary a readable specification, not just an implementation detail.

- **Server time authority** — `server_utc_time_at_request` is surfaced in the `HTTP 403` body for debugging transparency. There is no request field for client-supplied time, making the time authority architecturally unambiguous.
- **Order identity and timestamps** — `order_id` and `created_at` are server-generated. The client cannot supply, predict, or influence either value.

4. UML Diagrams

Rubric Criterion: Correct System Sequence Diagrams (SSD) showing happy and failure paths, and an Activity Diagram with explicit code decision points. Notation must be accurate.

4.1. Diagram 1: System Sequence Diagram — Secure Checkout (UC-5)

Engineering Description

This SSD models the complete server-side execution pipeline for **UC-5: Secure Checkout & Payment**, capturing the interaction between four participants: the **Customer (Browser)**, the **System (API Server)**, the **Database**, and the **Payment Gateway**.

The diagram is structured as a single outer `alt/else` cascade, with each `alt` branch capturing one failure gate in the exact order the server evaluates it:

1. **`REQ21`** — Empty cart guard returns `HTTP 400`.
2. **`REQ18`** — Payload field-length validation returns `HTTP 422`.
3. **`REQ15`** / **`REQ20`** — Pre-checkout stale-cart refresh; unavailable item block returns `HTTP 422`.
4. **`REQ17`** — Server-side price recalculation (client total discarded silently; annotated explicitly in the diagram).
5. **`REQ19`** — Operating-hours gate using server UTC clock returns `HTTP 403`.
6. Payment Gateway decline returns `HTTP 402`.
7. **Happy path** — order persisted and `HTTP 201` returned. **`REQ16`** is annotated on this terminal response.

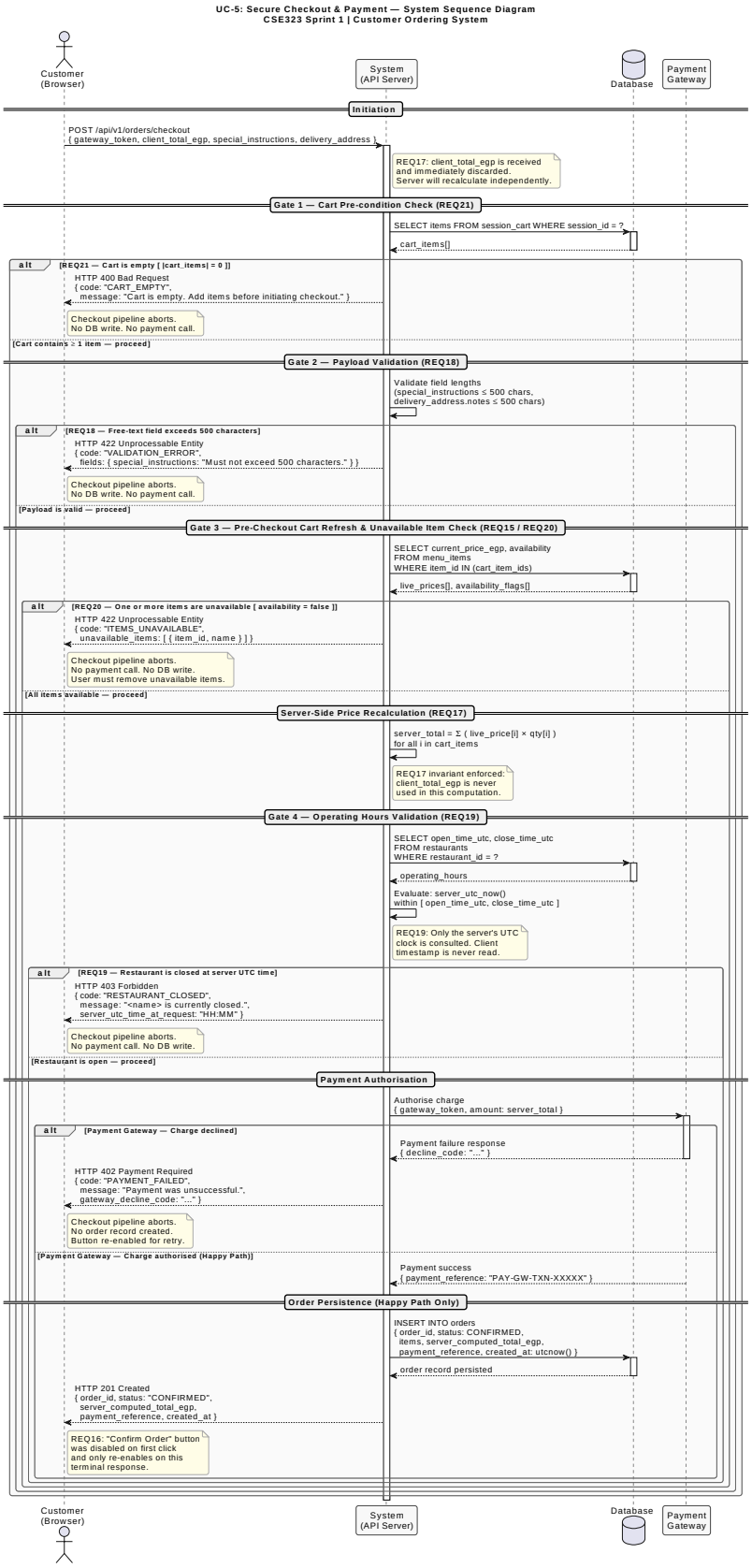

Figure 1: SSD — Secure Checkout & Payment (UC-5): Happy Path and all Failure Paths

4.2. Diagram 2: Activity Diagram — Add Item to Cart Logic (UC-3)

Engineering Description

This activity diagram models the complete **server-side execution logic** for **UC-3: Add Item to Cart**, structured across two swim lanes — **Browser (Client)** and **API Server** — with **Database** operations annotated as interaction steps within the server lane.

Four explicit decision diamonds correspond to the four padlock guards from the D2 Requirements Report. Each guard appears in the exact order the server evaluates it, forming an early-exit chain where any failed guard immediately returns an error response and terminates the flow:

1. **Padlock 1** (`REQ12`/`REQ13`) — Is quantity a strict positive integer? → `HTTP 422` on failure.
2. **DB Lookup 1** — Resolve `item_id`; does item exist? → `HTTP 404` on failure.
3. **Padlock 2** (`REQ20`) — Is `item.availability = true`? → `HTTP 422` on failure.
4. **DB Lookup 2** — Fetch active session cart.
5. **Padlock 3** (`REQ14`) — Is cart empty, or does item belong to the same restaurant? → `HTTP 409` on conflict.
6. **Success path** — Write line item at DB-authoritative price; return `HTTP 200` with updated cart. Price is always sourced from the database record loaded in DB Lookup 1; the client never supplies a price.

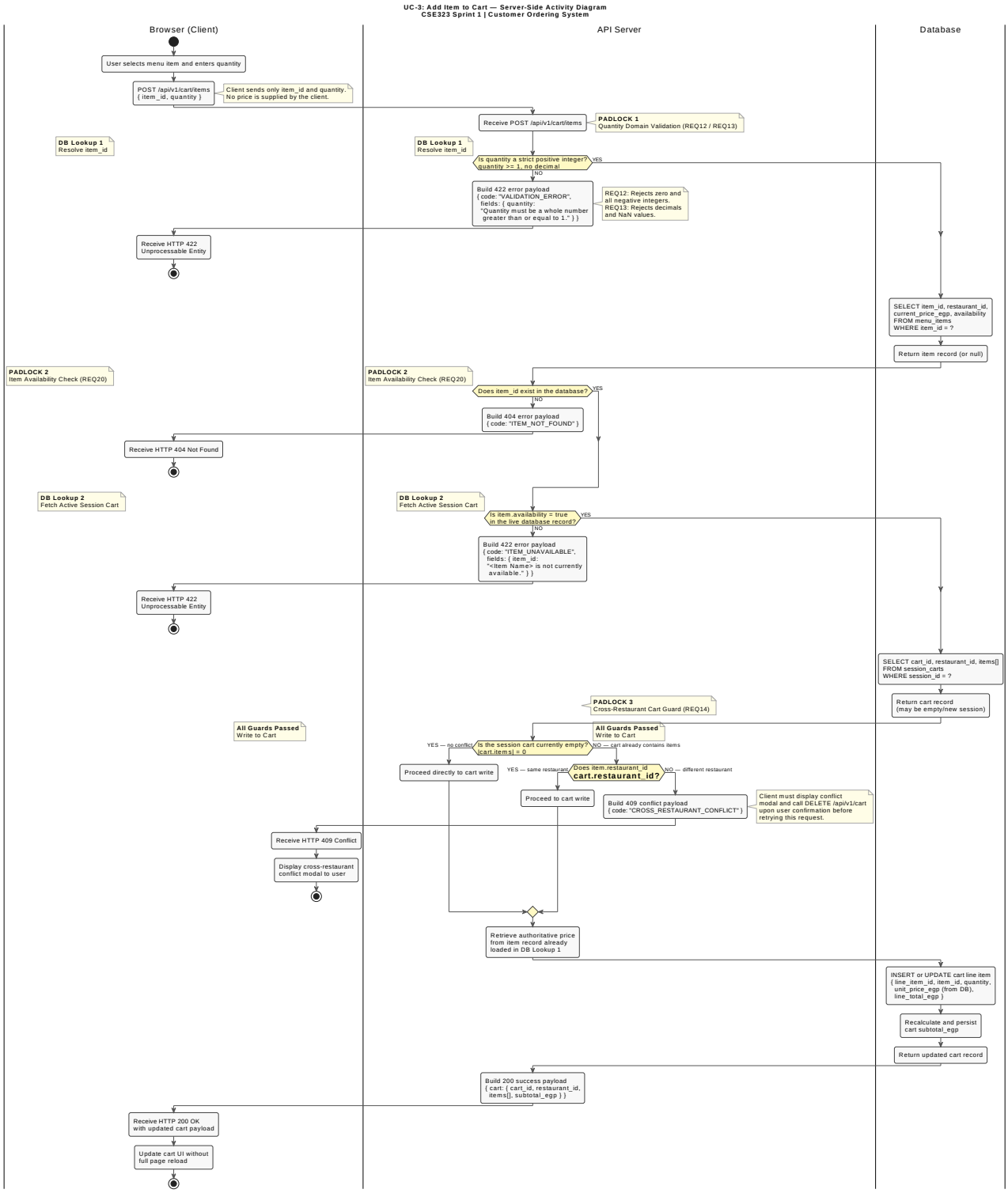

Figure 2: Activity Diagram — Add Item to Cart Server-Side Logic (UC-3), showing all four Padlock guards as explicit decision diamonds

End of Deliverable D3 — Design Specification Report

CSE323 Software Engineering, Spring 2026 — Egypt-Japan University of Science and Technology